

B2 group 1 project report

Contents

1	Introduction	3
2	Datasets	4
2.1	Datasets Used	4
2.2	Dataset's core Problem	4
2.3	Solution	4
3	2 Stages Model, Phase 1: Classifier	7
3.1	Data Preparation	7
3.2	Centralized Training Setup	8
3.3	Problems/Issues	8
4	2 Stages Model, Phase 2: Object Detection	9
4.1	Federated Training	9
5	Organization	10
5.1	Project Management	10
5.2	Github Usage	10
6	Results	11
6.1	Results	11

1 Introduction

Computer vision is a field of artificial intelligence (AI) that uses machine learning and neural networks to teach computers and systems to derive meaningful information from digital images, videos and other visual inputs—and to make recommendations or take actions when they see defects or issues. [3].

In this project our task was to use an already existing dataset to fine tune object detection model to recognize custom classes and efficiently deploy it on Edge Devices (for example Jetson Nano). these lightweight deployments are crucial for real-time performance and inference.

What makes this topic particularly interesting is that it brings AI into the real world and shows how useuful it can be, especially at a time when public discourse around AI often centers on its risks and negatives. unlike abstract generative models, object detection has clear benefits across a varied range of domains like medicine, autonomous vehicles, traffic safety and so on. Moreover, computer vision sits at the intersection of many sciences, it takes inspiration from biology, by modeling how visual systems take and process information, mathematics, via linear algebra, statistics, and neuroscience, via structure of artificial neural networks that mimic our brains. Advancements in computer vision not only enhance technological capabilities but also can make us understand biological vision systems. in this way, progress in AI and neuroscience can mutually reinforce each other, potentially leading to breakthroughs in both artificial and human intelligence.

2 Datasets

2.1 Datasets Used

Due to the size of the group, it is decided that we (Group 1) shall be working with a given dataset from the AI City Challenge 2023 Second Track [1].

The dataset contains 14 classes with sufficient training data. These classes are:

- | | |
|-----------------|-------------|
| 1. bus | 8. sedan |
| 2. coupe | 9. suv |
| 3. crossover | 10. taxi |
| 4. hatchback | 11. truck |
| 5. jeep | 12. van |
| 6. mpv | 13. vehicle |
| 7. pickup-truck | 14. wagon |

2.2 Dataset's core Problem

Our dataset had a core problem, it was not compatible to standard object detection models like Yolo. The main issue was, that out of X cars on the picture, we only had labels for 2 or 3 per frame, all the other cars were unlabeled and during training it caused significant issues. Even when our model was guessing correctly, since we didn't have any labels for other cars, it was considered as background and because of that our model was getting punished for correctly assessing an image, it was reducing its confidence and accuracy. We realised that training on such dataset would bring us nowhere and thus started looking and thinking for possible solutions.

2.3 Solution

One of the possible solution for this would be to find a more appropriate data set with more than just one or two objects in each frame of the dataset. However ultimately we decided to go with pseudo labeling method, which will attempt to turn the existing dataset to a more suitable one.



Figure 1: Example of how original dataset looked like.

The idea is using a base object detection model (for example yolo11n) for detecting only vehicles and putting the bounding box on them, after that a classifier (we train it ourselves, or a pretrained one can also be used) will be used to generate labels on those bounding boxes and at the end, we can run inference on the original video and extract the annotated frames, after that we can get dataset with multiple cars annotated and thus it will be suitable for fine-tuning the yolo11n object detection model.

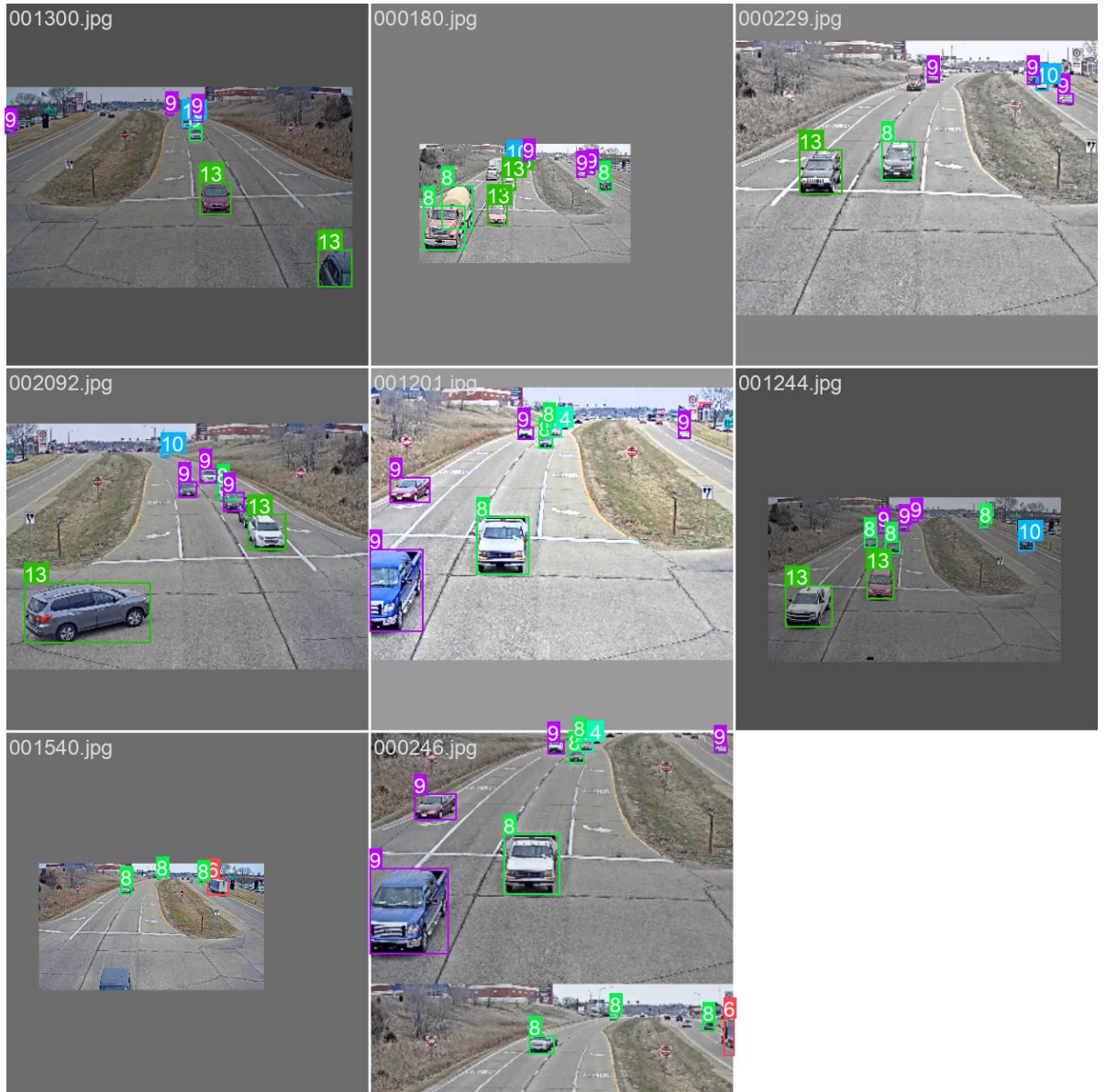


Figure 2: batch from newly created dataset

3 2 Stages Model, Phase 1: Classifier

3.1 Data Preparation

To prepare for the fine-tuning of a classifier, you will need only the picture of the objects and neither the bounding box nor coordinates, because in our case all the images were centered. The original frames from the dataset, which contained some bounding box of one or two cars, is cropped so that only those cars are visible in the image. For example, if the frame has two annotated cars, two standalone pictures of those cars will be generated. Furthermore, those processed car images will then be categorized into 14 different classes, hence 14 different folders based on the labels given by the original dataset.

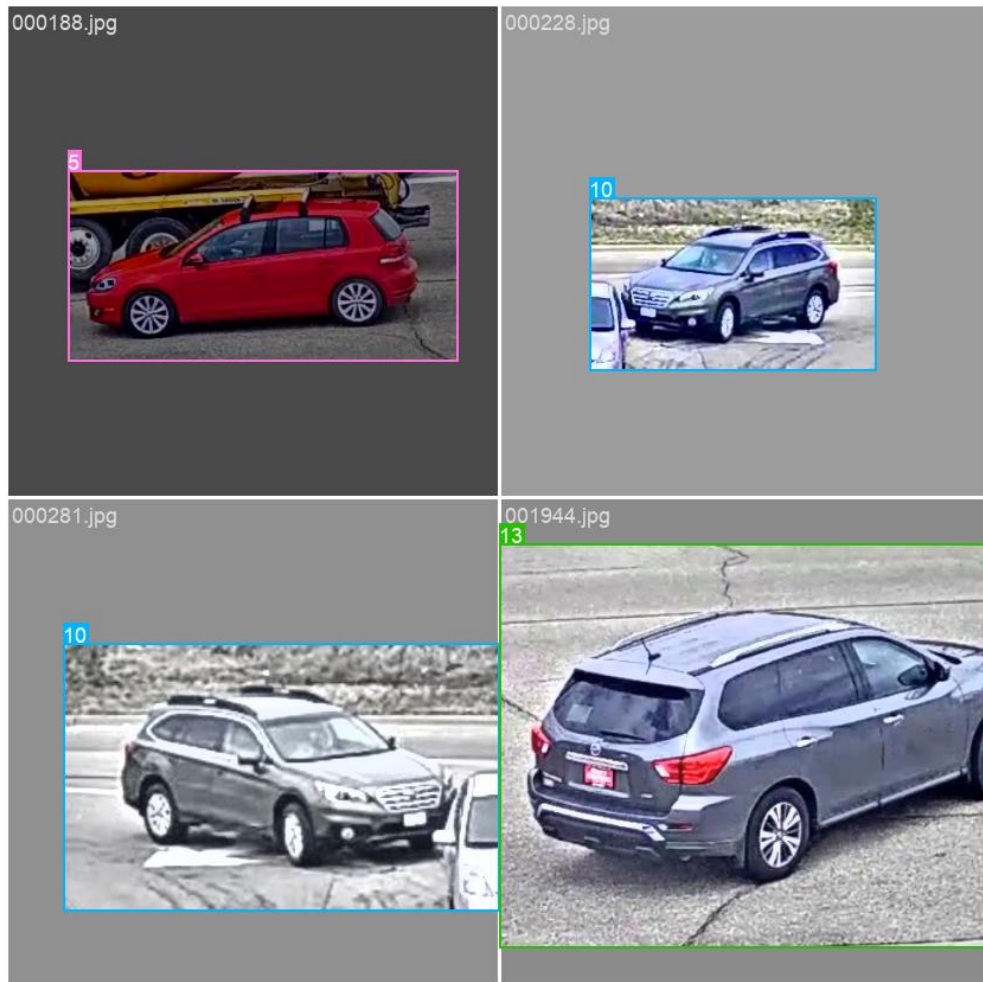


Figure 3: batch from classifier

There was a problem stemming from extracting the frames of video footages and using it as dataset, which is that there will be a lot of pictures, where the changes in the scene are so minimal, they can be thought of as duplicates. For example in the dataset, one fifth of the dataset is of the same car with the same angle and minimal background changes. To solve this problem, every five images are chosen as part of the dataset.

3.2 Centralized Training Setup

We decided to train the classifier centrally due to time constraints. And since we are using ultralytics, most of the coding process has already been handled and we only need to call the Ultralytics-Yolo API.

3.3 Problems/Issues

It is known that the quality of one's model is reflected through the quality of the dataset that it is trained on. And thus our object detection model, which is our main product, can only be as good as our classification model.

After training the classifier model on 100 epochs centralized, the metric for the classification is quite positive and when checking the training validation images, the prediction is correct 85 to 90 percent. However when running the inference with yolo11n object detection model for dataset extraction, we encounter two critical problems.

Firstly the ultralytics object detection is not always correct. For instance, there were few cases where the yolo model put a bounding box on the street light while that was not a vehicle and the classifier did not annotate it for instances. Secondly while the classifier model is accurate, if the objects are too far away, the prediction is very jittery and only stabilizes when the car is close to the camera and clear looking. We could raise the confidence threshold of the classifier and/or yolo11n, however through the testing of those cases, minimal objects were detected and the jittery though minimal still persists. By taking each every five frames, we hope to minimize this problem, however it is not at all a reliable method. And thus one can predict that the object detection model will possibly be more jittery and less quality than the two models combined that we have at this stage.

4 2 Stages Model, Phase 2: Object Detection

4.1 Federated Training

For this training we use the Flower [2] because it's a well known and well documented framework for Federated Learning. We want to simulate the real-life where the edge devices are far apart, or at least on the same WLAN, and so we have set up a remote server using Google Cloud to act as the central server for our clients (edge devices).

As for the hyper parameters of the training:

- **imgsz**: the image size we pick is 416 by 416 (**imgsz=416**), but we should have used 640 which is a more standard size of the dataset of that is used to train the yolo model.
- **batch**: while batch size of 1–32 is normally considered as small batch size, with the constrain of the server as well as devices (which will be discussed later) on non cpu devices, 2–4 is pushing the limit of cpu devices, 8–16 seems to be the ideal one without the epoch feeling too long.
- We also enable parameters such as: **save**, **plot**, **val** in order to have the metrics and its graphs saved.
- The parameter **worker** refers to the speed that the dataset being loaded for the training.
- On the topic of limited memory usage, **max_det** limits the prediction that the model makes while training.

Otherwise, parameters that we are not comfortable with changes, we set it to the ultralytics default.

Finally on the server, one can set off the training and **fedavg** is what we used, which synchronously take the training updates from all the clients and aggregates it. Furthermore there were quite a critical problem with the server that we found during the training that was not our fault: if the local epochs take more than 4 minutes to be done to send the update back to the server, Google Cloud will disconnect it, making the aggregated model lacking that client's update and we have to reconnect the clients manually. But due to the lack of human resources and edge devices, we decided not to create a server

of our own and train it centralized. However if anyone is able to create their own remote server, our setup can work perfectly.

5 Organization

5.1 Project Management

Due to limited number of participants in our group, we decided to take the dynamic approach. We supported each other throughout every task. While individual members may have taken lead on specific components (for example Dat: flower, federated learning, server/client architecture and Luka: Data preparation and processing, optimization and deployment on edge devices), these tasks were only completed successfully thanks to the mutual support and collaboration within the team. this flexible workflow allowed us to quickly adapt to problems and maintain consistent progress throughout this project.

5.2 Github Usage

<https://github.com/Lukamanvelidze/B2-EdgeAI> is the link to our github main branch where everything code related is explained and we also provide a detailed instructions on how to use and reproduce each step mentioned in this document. There are in total five main folders, with some of the folders have subfolders in them(detailed descriptions can be found on github):

1. data processing
2. train
3. inference
4. trained model
5. documentation

The first one contains codes that help with the data preperation for the training of the classifier as well as the object detection model. The second one is the server-client setup of the federated learning (it doesn't contain our centralized learning code since it's simply calling the Ultralytics API). The third one is as the name of the folder: for inference. Then comes the trained model as well as its different data format: .pt, .onnx and .engine. And finally is the folder containing this project's report.

6 Results

6.1 Results

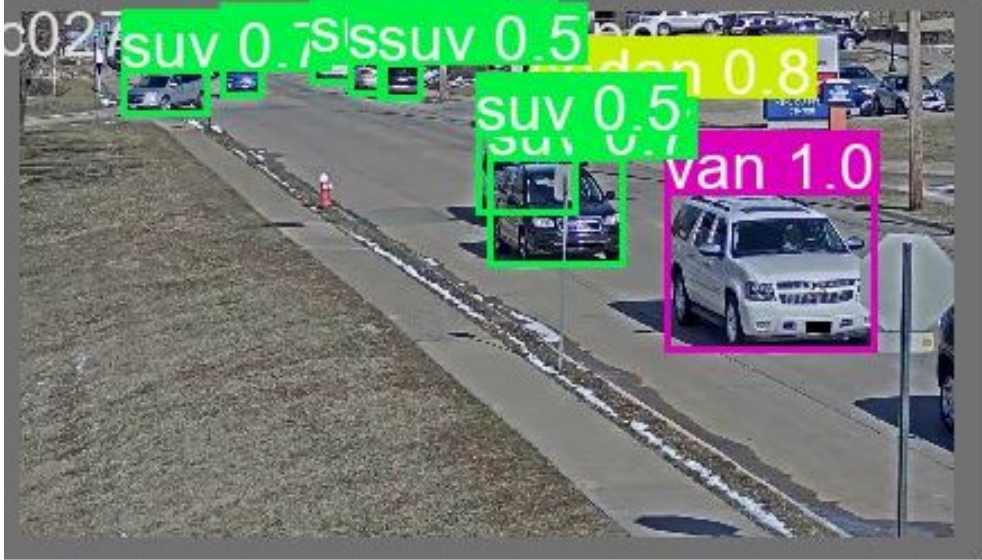


Figure 4: results

As you can see from figure 4, a substantial amount of cars are being detected with the confidence threshold of 0.5 and seemingly accurate as well. However this characteristic seems to only appear when it is done on pictures and videos from the AI City challenge tracks. When we take video on the street of Berlin ourself and run inference on it, the result is a lot of jittery, meaning the label of an object was switching between classes quickly. Though this problem could be attributed to a foreign environment, for instance dataset for the color taxi is only green while in Berlin we observed its yellow, or another type of car that is not in our defined class. One final worth noting characteristic of this model is that the closer the object to the point of view, the less jittery and more stable the prediction of the object. With the way we generate the dataset using the classifier and base yolo11n (confidence threshold of 0.79), this makes sense.

With our inference code (can be found on our github) and different model types on different type of edge devices, we can see that even though the inference speed of the .engine is the superior to .onnx and .pt, the total latency on a Jetson Nano with GPU used is far slower than on a Thinkpad T480 on

CPU with .onnx. And so this is more on the optimality of our inference code as well as the specs of the edge devices.

References

- [1] Ai city challenge 2023 tracks. <https://www.aicitychallenge.org/2023-challenge-tracks/>, 2023. Accessed: July 30, 2025.
- [2] Flower ai. <https://flower.ai>, n.d. Accessed: July 30, 2025.
- [3] IBM. What is computer vision? <https://www.ibm.com/think/topics/computer-vision>, n.d. Accessed: July 30, 2025.